

■ Lab 5 — Outputs & Variables

■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Organisé votre code Terraform selon les **bonnes pratiques**
 - Exporté des attributs de ressources avec les **outputs**
 - Déclaré et utilisé des **variables** avec différents types
 - Assigné des variables via **CLI**, **tfvars** et **variables d'environnement**
 - Ajouté des **règles de validation** sur les variables
-

■ Étape 1 — Structure et fichiers

```
cd labs/lab5-outputs
```

Créez les fichiers suivants :

`backend.tf`

```
terraform {
  backend "s3" {
    bucket      = "tf-training-<votre-prenom>-982908300187"
    key         = "terraform.tfstate"
    region      = "eu-west-1"
    use_lockfile = true
    encrypt     = true
  }
}
```

■ ■ Remplacez `` par votre prénom en minuscules sans accent. Ex : `tf-training-alice-982908300187`

`versions.tf`

```
terraform {
  required_version = ">= 1.15.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

`provider.tf`

```
provider "aws" {
  region = var.region
}
```

`variables.tf`

```
variable "username" {
  description = "Votre prenom - permet de differencier les ressources entre participants"
  type        = string

  validation {
    condition     = length(var.username) >= 2 && length(var.username) <= 20
    error_message = "Le username doit contenir entre 2 et 20 caractères."
  }
}

variable "region" {
  description = "Region AWS de déploiement"
  type        = string
  default     = "eu-west-1"
}

variable "instance_type" {
  description = "Type d'instance EC2"
  type        = string
  default     = "t2.micro"

  validation {
    condition     = contains(["t2.micro", "t2.small", "t2.medium"], var.instance_type)
    error_message = "Le type d'instance doit être t2.micro, t2.small ou t2.medium."
  }
}

variable "environment" {
  description = "Environnement de déploiement"
  type        = string
}
```

```
default      = "dev"

validation {
  condition     = contains(["dev", "prod"], var.environment)
  error_message = "L'environnement doit être dev ou prod."
}
}
```

`main.tf`

```
resource "aws_security_group" "lab5_sg" {
  name          = "lab5-sg-${var.username}-${var.environment}"
  description    = "Security group Lab 5 - ${var.username}"

  tags = {
    Name       = "lab5-sg-${var.username}"
    Lab        = "lab5"
    Username   = var.username
    Environment = var.environment
  }
}

resource "aws_instance" "lab5_instance" {
  ami                = "ami-0694d931cee176e7d"
  instance_type      = var.instance_type
  vpc_security_group_ids = [aws_security_group.lab5_sg.id]

  tags = {
    Name       = "lab5-instance-${var.username}"
    Lab        = "lab5"
    Username   = var.username
    Environment = var.environment
  }
}
```

`outputs.tf`

```
output "instance_id" {
  description = "ID de l'instance EC2"
  value      = aws_instance.lab5_instance.id
}

output "instance_public_ip" {
  description = "IP publique de l'instance"
  value      = aws_instance.lab5_instance.public_ip
}

output "security_group_id" {
  description = "ID du Security Group"
  value      = aws_security_group.lab5_sg.id
}

output "security_group_name" {
  description = "Nom du Security Group"
  value      = aws_security_group.lab5_sg.name
}

output "deployment_summary" {
  description = "Résumé du déploiement"
  value = {
    username    = var.username
    environment = var.environment
    instance    = aws_instance.lab5_instance.id
    region      = var.region
  }
}
```

■ Étape 2 — Les 3 façons d'assigner les variables

Méthode 1 — CLI flag (`-var``)

```
terraform init
terraform apply -var="username=<votre-prenom>" -var="environment=dev"
```

Méthode 2 — Fichier `tfvars`

Créez `terraform.tfvars` :

```
username    = "<votre-prenom>"
environment = "dev"
instance_type = "t2.micro"
```

```
terraform apply
```

■ Terraform charge automatiquement `terraform.tfvars` s'il est présent. Pour un fichier différent : `terraform apply -var-file="prod.tfvars"`

Méthode 3 — Variables d'environnement (`TF\_VAR\_`)

```
export TF_VAR_username="<votre-prenom>"
export TF_VAR_environment="dev"
export TF_VAR_instance_type="t2.micro"

terraform apply
```

■ Le préfixe `TF\_VAR\_` suivi du nom exact de la variable est reconnu automatiquement par Terraform.

■ Étape 3 — Tester la validation

```
# Valeur d'environnement invalide
terraform plan -var="username=<votre-prenom>" -var="environment=staging"
```

Résultat attendu :

```
Error: Invalid value for variable
  L'environnement doit être dev ou prod.
```

```
# Type d'instance invalide
terraform plan -var="username=<votre-prenom>" -var="instance_type=t3.large"
```

Résultat attendu :

```
Error: Invalid value for variable
  Le type d'instance doit être t2.micro, t2.small ou t2.medium.
```

■ Étape 4 — Utiliser terraform output

```
# Tous les outputs
terraform output

# Un output spécifique
terraform output instance_id

# Format JSON (utile pour les scripts)
terraform output -json deployment_summary

# Format brut sans guillemets (utile en CI/CD)
terraform output -raw instance_id
```

■ Les outputs peuvent exposer un **attribut spécifique**, un **objet complet**, ou être utilisés par d'autres modules.

■ Étape 5 — Nettoyage

```
terraform destroy -var="username=<votre-prenom>"
```

■ Validation du Lab

#	Critère	Validé
1	6 fichiers séparés créés (`backend.tf`, `versions.tf`, `provider.tf`, `variables.tf`, `main.tf`, `outputs.tf`)	■
2	Validation bloque `environment=staging`	■
3	Validation bloque `instance_type=t3.large`	■
4	`terraform apply` via `-var` fonctionne	■
5	`terraform apply` via `terraform.tfvars` fonctionne	■
6	`terraform apply` via `TF_VAR_` fonctionne	■

#	Critère	Validé
7	`terraform output -json deployment_summary` affiche un objet JSON	■
8	`terraform destroy` supprime les ressources	■

■ ■ Résolution des problèmes courants

Problème	Cause	Solution
`No value for required variable`	Variable sans default non fournie	Ajouter `-var="username=..."` ou `terraform.tfvars`
`Invalid value for variable`	Validation échouée	Vérifier la valeur fournie
`TF_VAR_` ignoré	Mauvais nom de variable	Vérifier l'orthographe exacte

■ ****Fin du Lab 5 — Votre code est bien structuré et dynamique !****

*Le Lab 6 introduit les ****locals**** pour simplifier et centraliser les expressions dans votre code.*